

VNUHCM - UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

CSC14003 – INTRODUCTION TO DATA SCIENCE



COURSE MILESTONE 1

CLASS 23KHDL02 - GROUP 4

Students:

23127168 – Cao Trần Bá Đạt
23127238 – Trần Hoài Thiện Nhân
23127516 – Bùi Nam Việt

Lecturers:

Mrs. Nguyễn Ngọc Thảo
Mr. Huỳnh Lâm Hải Đăng
Mr. Nguyễn Thanh Tình

November 15th, 2025

Contents

1	Introduction	4
2	Implementation Process	5
2.1	Libraries and tools	5
2.2	Download source files for paper's version and metadata	5
2.3	Extract and delete	6
2.3.1	Detect and fix file type	6
2.3.2	Extract and clean	6
2.4	Scraping references	7
2.5	Method of handling last id of month	7
2.6	Pipeline	8
3	Summary	8
3.1	Photo evidence	9
3.2	Relevant Statistics	9
3.2.1	Running Time	10
3.2.2	Memory Footprint	10
4	References	11
	Tài liệu tham khảo	11

List of Figures

1	Start run evidence	9
2	End run evidence	9

Acknowledgment

To complete this milestone, we would like to send my sincere and deepest thanks to the teachers who have wholeheartedly guided and imparted knowledge to me throughout the past time. First of all, we would like to thank the main lecturer, Ms. Nguyen Ngoc Thao and then the two instructors of the project, Mr. Huynh Lam Hai Dang and Mr. Nguyen Thanh Tinh guided us through the tools, direction, and methods to approach this milestone easily and in the right direction.

This milestone has helped us learn and implement how to scrape websites through libraries and APIs as well as how to handle common problems when scraping. In addition, programming skills with interacting with multi-threaded systems and file processing have also been acquired through this milestone.

1 Introduction

This report summarizes the execution of Milestone 1 for the "Introduction to Data Science" course, which aimed to apply data scraping knowledge to a practical project.

The primary task was to build a scraper for the arXiv repository. As required, we performed the following tasks for an assigned range of paper IDs:

- Collected all available versions of each paper.
- Downloaded the full LaTeX (TeX) source files and removed image files to reduce storage.
- Extracted metadata (title, authors, dates), BibTeX files, and crawled reference information.
- Utilized suggested tools such as the arXiv API and the Semantic Scholar API for data collection.

This report details the implementation process, the tools used, and the performance evaluation of the scraper.

Group demo video link: <https://www.youtube.com/watch?v=jVv241fCPLA>

2 Implementation Process

2.1 Libraries and tools

- arxiv: library acts as a single bridge between your Python code and the arXiv server.
- request: library, use for calling the scholar semantic API
- Semantic Scholar: acts as a knowledge database to search for references.
- os: library, used to create folder.
- re: the library, used for extracting text and file names.
- json: library, used for reading and writing json files.
- time: library, use for sleeping.
- shutil: module, used for performing operations on files and directories.
- tarfile: library, use to handle .tar .gz.
- subprocess: library, used to run external command. Use subprocess.run to call the system file command, to "check" the downloaded file
- gzip: module, used to unzip the folder.
- ThreadPoolExecute, as _complete: class and function, used to split into multiple parallel streams.
- Lock: class, used to ensure safety when running multithreaded.
- psutil: library, used to measure system resources.
- memory_profiler: library, used to measure memory.
- sys: library, used to interact with the system.

2.2 Download source files for paper's version and metadata

- To access the paper we use the arxiv library by initializing a client to communicate with the arXiv server API.

- Search for pages by their id using the arxiv library commands and get an object containing the article's metadata.

- From this object you can call its fields such as title, author, submission date, publication venue through fields of the object

- The key point here is that when searching by just the article id, calling the `entry_id` field of the article object will return the article id but with the latest version name at the end. So we will use regex to find the latest version name from here.

- We will use a loop to iterate from version 1 to the latest version by accessing each version to get the object as above.

- By this loop we will get the releases of each version and the sources for each version through the fields of the object.

2.3 Extract and delete

2.3.1 Detect and fix file type

Execute the `run` function file with the downloaded source file to capture the string that the file command prints out to see what the nature of that file is.

- If this run results in a **“PDF document”**, then it is a PDF.
- If this result is **“gzip compressed data”** then it is likely that the author only has a single `.tex` file and performs a regex search for the file name. If it is found, it will return the file type `gz` and the file name `tex`. If not, it will return the file type **.tar.gz**.
- If this results in **“tar archive”** then this is a standard `.tar.gz` archive so it will return `tar.gz`.
- The remaining results will return the file type as unknown.

2.3.2 Extract and clean

Perform the call to the `“detect and fix file style”` function as mentioned above to get the file type.

- If it is **“pdf”**, it reports success and stops immediately (because there is nothing to extract).
- If it is **“unknown”**, it reports failure and stops.
- For `tar.gz` files use `“tarfile.open”` and `“tar.extractall”` functions to extract all files into a subdirectories
- For `gz` files use the **“gzip.open”** function to unzip that single file into a subdirectory

After decompressing in the above step, the function continues to clean up unnecessary files.

- Directory Scan: It uses `os.walk` to walk through all the files just extracted in the subdirectories.
- It checks whether the file name (in lowercase) ends with `.tex` or `.bib`.
- If the file is not `.tex` or `.bib` (e.g. `.jpg`, `.png`, `.sty`, `.log`, `.pdf` included...), it will be considered a "trash" file and deleted with `os.remove`.
- Counts the number of deleted files and saves it to the `deleted` variable.

After executing this function, the source file folder that was originally downloaded will be deleted.

2.4 Scraping references

- Use the url containing the path of the instructor's provided scholar semantic API and put the id of the article at the end. We also use the S2 API Key:

cf6G5yldwF4UEzswq3WKX72B6uffqNv17LQDo8Oi which we obtained from Semantic Scholar at 2:40 PM on November 14, 2025 and

a8okwqTLp18Ku1vBXJ1Jb6eRoDKpmAem41VjtFCY which we obtained from Semantic Scholar at 2:11 PM on November 14, 2025.

- Use `request` to make an API call with the following params in code to get the citations of the article.

- There will be cases where the API will give error 429, this error is quite common so the id will be queued for later execution.

- Get the list of citations and filter out the citations that are only in ArXiv.

- We will iterate through the list of references and then use the "**get**" function to get the references that are external and only on the arxiv page.

- From the filtered references, perform the `get` function to get the required information such as id, paper title, authors, submission date, semantic scholar id.

- Save the results of an article into a dictionary and write it to a json file.

2.5 Method of handling last id of month

When we start running the first id of the first month we will use a regular step like this:

- **Step 1:** Get the first id of the month that is starting and add it to 50, then directly access the article page corresponding to the id that has just been added via the

http link. If the result still exists, continue adding 50 and repeat step 1. Otherwise, go to step 2.

- **Step 2:** Take the id of the previous step to subtract 10 and then check if the article exists like the step above. If the article does not exist, repeat step 2. If the id exists, go to step 3.
- **Step 3:** Similar to step 1 but this time add 5.
- **Step 4:** Similar to step 2 but this time minus 1.

After running the above loop, we will find the exact last id of the month.

2.6 Pipeline

- Initially we will create from input input is month and id as standard id.
- If the input has 2 different months, handle as in 2.5.
- With an input range of 2 months, we will divide it into 2 runs: once in the first month and once in the last month.

For each article id:

- Search for articles through the arxiv library and get the sources and metadata. One thing to note in this step is that when getting the source of an article, we will immediately unzip and delete unnecessary files, instead of having to download the entire article and then unzip and delete.
- Make a call to the Semantic Scholar API to get the references.

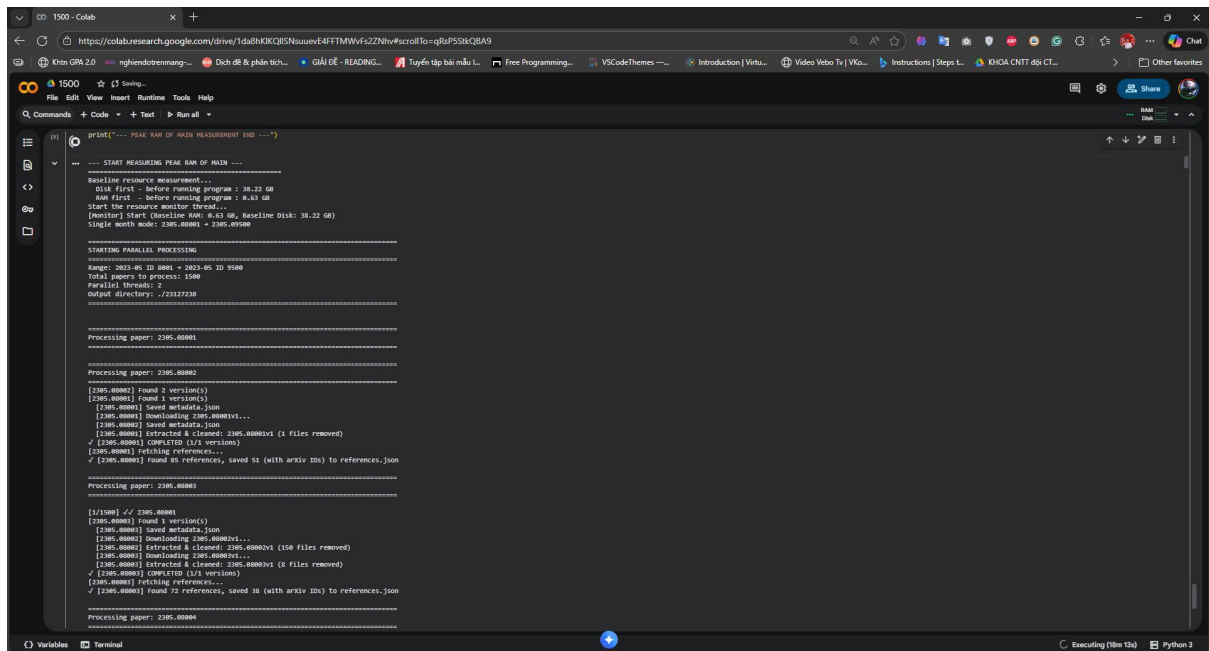
Notice: When executing the .ipynb file, it will create 3 .py files to run the program on these 3 files.

3 Summary

We have successfully collected all 15000 articles and the statistics below are from 10% of the articles corresponding to 1500 articles.

The data collection run was exceptionally successful. A total of 1500 papers were processed with a 100% success rate for both arXiv source crawling and Semantic Scholar reference extraction. The scraper demonstrates remarkable efficiency, processing the entire 1500 paper workload in more 2 hours. The most impressive finding is the resource footprint: with a peak RAM usage of only 122.82 MiB (measure by memory profiler), the script proves to be extremely lightweight and scalable. The final 2.36 GB dataset is clean and robust, validating the project's data collection methodology.

3.1 Photo evidence



```
print("... PEAK RUN OF MAIN MEASUREMENT END ...")

--- START MEASURING PEAK RUN OF MAIN ---

Baseline resource measurement...
disk first - before running program : 38.22 GB
ram first - before running program : 8.61 GB
Start the resource monitor thread...
[monitor] start (baseline ram: 8.61 GB, baseline disk: 38.22 GB)
Single month mode: 2385.08081 + 2385.09590

STARTING PARALLEL PROCESSING
Range: 2023-05-10 0801 + 2023-05-10 0908
Total papers to process: 1500
Parallel threads: 2
output directory: ./23127238

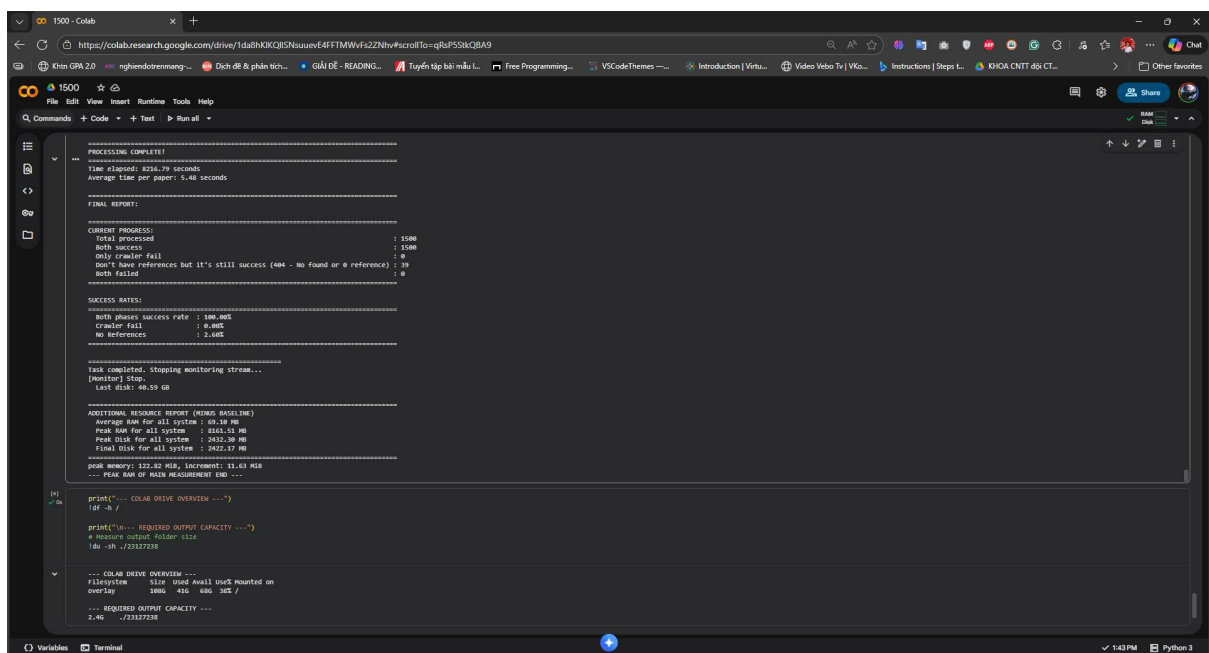
Processing paper: 2385.08081

Processing paper: 2385.08082
[2385.08082] found 2 version(s)
[2385.08082] saved metadata.json
[2385.08082] downloading 2385.08082v1...
[2385.08082] saved metadata.json
[2385.08082] extracted & cleaned: 2385.08082v1 (1 files removed)
✓ [2385.08082] converted (1/1 version(s))
[2385.08082] fetching references...
✓ [2385.08082] found 51 references, saved 51 (with arxiv IDs) to references.json

Processing paper: 2385.08083
[2385.08083] found 3 version(s)
[2385.08083] saved metadata.json
[2385.08083] downloading 2385.08083v1...
[2385.08083] extracted & cleaned: 2385.08083v1 (150 files removed)
[2385.08083] downloading 2385.08083v2...
[2385.08083] extracted & cleaned: 2385.08083v2 (8 files removed)
✓ [2385.08083] converted (1/2 version(s))
[2385.08083] fetching references...
✓ [2385.08083] found 72 references, saved 38 (with arxiv IDs) to references.json

Processing paper: 2385.08084
```

Figure 1: Start run evidence



```
PROCESSING COMPLETE!
Time elapsed: 8256.79 seconds
Average time per paper: 5.48 seconds

FINAL REPORT:

CURRENT PROGRESS:
Total processed : 1500
Both success : 1500
Only crawler fail : 0
Don't have references but it's still success (no - no found or a reference) : 0
Both failed : 0

SUCCESS RATES:
Both phases success rate : 100.00%
Crawler fail : 0.00%
No references : 2.66%

Task completed. Stopping monitoring stream...
[monitor] stop.
Last disk: 40.59 GB

ADDITIONAL RESOURCE REPORT (BASIC BASELINE)
Average ram for all system : 69.18 MB
Peak ram for all system : 8433.50 MB
Peak disk for all system : 3832.00 MB
Final disk for all system : 2422.17 MB

peak memory: 122.82 MB, increment: 11.63 MB
--- PEAK RUN OF MAIN MEASUREMENT END ---

print("... COLAB DRIVE OVERVIEW ...")
(df > .)

print("... REQUIRED OUTPUT CAPACITY ...")
# measure output folder size
lsdu -sh ./23127238

... COLAB DRIVE OVERVIEW ...
filessystem size used avail used% mounted on
overlay 180G 41G 60G 30% /

... REQUIRED OUTPUT CAPACITY ...
2-MB ./23127238
```

Figure 2: End run evidence

3.2 Relevant Statistics

This section outlines the key metrics related to the data collection process from arXiv and Semantic Scholar.

- **Number of Papers Scraped Successfully:** 1500
- **Range ID:** 2305.08001 - 2305.09500
- **Time:** 18:50 on 15/11/2025 to 21:07 on 15/11/2025
- **Overall Success Rate:** 100%
- **Average Paper Size (Before Cleaning):** 2.4 MB
- **Average Paper Size (After Cleaning):** 1.5 MB
- **No references:** 39
- **No reference rate:** 2.6%
- **Reference Metadata Scrape Success Rate:** 100%

Statistical Analysis and Commentary:

The 100% success rates for both the main scraper and the reference extractor are outstanding. This indicates that the error handling, request logic (including requests with a User-Agent), and API key integration (for Semantic Scholar) are robust.

The reason why no reference was found can be explained by the fact that when calling the Semantic Scholar API for the corresponding ID, it returns a 404 that the reference of the article was not found or that the article has no references.

Furthermore, the data cleaning step is highly effective. The reduction in average paper size from 2.4 MB to 1.5 MB represents a 37.5% reduction in storage. This cleaning process (removing non-essential image/style files) significantly optimizes the final dataset.

3.2.1 Running Time

- **Total Processing Time (1500 Papers):** 2 hours, 16 minutes, 57 seconds
- **Average Time per Paper:** 5.48 seconds

3.2.2 Memory Footprint

- Measure by memory profiler (measure ram when running main function)
 - **Maximum RAM Used:** 122.82 MiB
 - **Average RAM Used:** 11.63 MiB
- Measure overall system Ram of google colab
 - **Maximum RAM Used:** 8161.51 MB
 - **Average RAM Used:** 69.10 MB

- Measure memory disk
 - **Peak disk:** 2432.30 MB $\approx$ 2.37 GB
 - **Final Output Storage Size:** 2422.17 MB $\approx$ 2.36 GB

4 References

- [1] Semantic Scholar, “Semantic Scholar Academic Graph API”, <https://www.semanticscholar.org/product/api>
- [2] Semantic Scholar, “API Tutorial — Semantic Scholar API”, <https://www.semanticscholar.org/product/api/tutorial>
- [3] Semantic Scholar, “Graph API Documentation”, <https://api.semanticscholar.org/api-docs/graph>
- [4] arXiv, “arXiv API User’s Manual”, <https://info.arxiv.org/help/api/user-manual.html>
- [5] arXiv, “arXiv Open Access and API”, <https://info.arxiv.org/help/oa/index.html>